ARI5118 Deep Learning For Computer Vision

Topic: Backpropagation and Gradient Flow - Chain rule, computational graphs, gradient flow through convolutions, cross-entropy, softmax

1.Study Notes

To understand the fundamentals of back propagation and gradient flow in Convolutional Neural Networks, first the goal of the model needs to be defined. The aim of the model is to optimise the weights and biases to make the errors as small as possible. To be able to achieve this, one needs to understand how the error is computed before being able to use it for backpropagation.

Softmax & Cross-entropy Loss

The output of a CNN used for classification would not just be what it thinks the input image is but rather it gives a distribution of probabilities across all possible classes. For example the output of the architecture seen in Fig 1. would not just be 'Cat', the output would be '[Cat 5%, Dog 10%, Bird 85%]'. This is done through softmax. Softmax is the mathematical function used especially in classification problems where the raw prediction scores, called logits, are converted into probabilities, these probabilities always add up to 1.

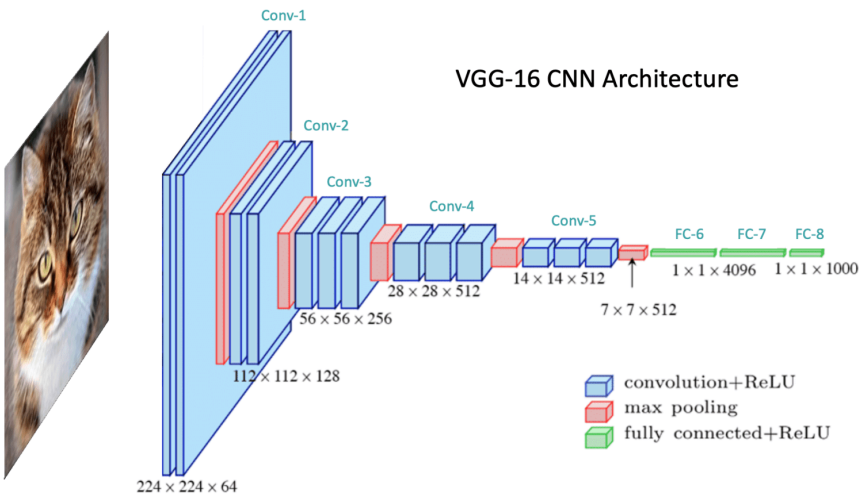

Figure 1 - Simple CNN Architecture

Fig 2. is an example of softmax, the output layer is passed through the activation function which outputs the probabilities. In the function the 'e' represents Euler's constant, which is approximately 2.718. This constant is used for a couple of reasons, the first being that it makes backpropagation computational cheaper and efficient. During backpropagation derivatives are constantly being calculated and the derivative of the e^X is e^X so using Euler's

constant prevents the formulas from becoming massive and complicated. The second reason is that neural networks can output negative numbers and since the probabilities can only be between 0 and 1 this does not work. Using Euler's constant ensure that the probabilities are always positive since e^X is always a positive number. Another reason 'e' is used is because the e^X is a continuously differential curve so the optimiser can make tiny adjustments to the weights without sudden jumps in probability. In the denominator of the function the summation is used to calculate the total sum of the exponential values so that the probability can be calculated.

Cross-entropy, also known as log loss, used the Softmax probabilities to determine how close those probabilities are to the actual label of the image. In the previously mentioned example if the model predicted [0.05 ,0.1, 0.85] and the true labels are [1,0,0], the cross entropy loss will penalise the model because the model places high probability in the incorrect class, i.e the model predicted the image as 'Bird' instead of 'Cat'.

y_i in Fig 3. is the one-hot encoded vector of the true label, i.e [1,0,0]. $\hat{y}_i$ is the output of the Softmax i.e [0.05,0.1, 0.85]. The log and the negative sign are used because the logarithm of numbers between 0 and 1 are always negative, so a negative of a negative is always positive. If the true label is 1 but the model predicts 0.05, the log(0.05) is a large negative number which indicates that the model is very wrong and a huge loss is produced. If the true label is 1 and the model is used to predict birds, a log(0.85) is a small negative number so the loss would not be as great.

Backpropagation

Backpropagation is the backward pass through a neural network used to calculate the gradient of the loss function with respect to the weights, which are then updated to minimise the loss. Before back propagation there are 2 steps, the forward pass and the loss calculation. The forward pass, also known as forward propagation, is the process in which the input is passed through the network to produce an output. In the convolutional layers the input is of shape (H,W,C) where W is the height, W is the width and C is the number of Channels of the image. In the convolutional layer as the filter slides over the input, each local region is multiplied element-wise with the filter and summed to produce a single value. A single bias term is then added to this value, so every position in the resulting feature map is shifted by the same bias. The filter moves across the input according to a stride, which determines how many pixels it shifts at each step, and padding may be applied to the input to control the spatial size of the output. That input is then passed to the activation function as can be seen in Fig 4. There are a number of different activation functions which can be used for CNNs, some of the most popular are ReLU, tanh, sigmoid and softmax.

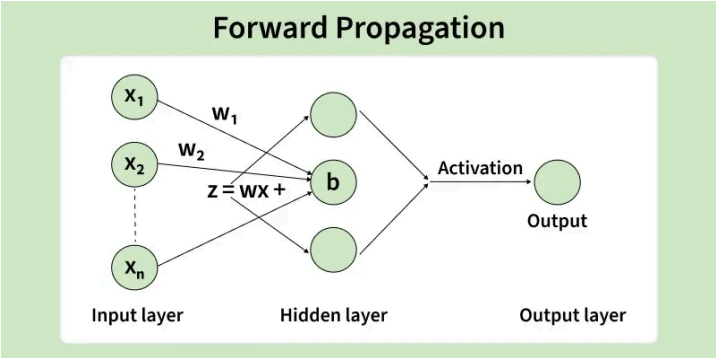

Figure 4 - Forward Propagation

Chain Rule & Gradient Flow

The Chain Rule is a quick and easy way to find the derivative of composite functions or nested functions. As can be seen in Fig 5. to differentiate f(g(x)) with respect to x, take the derivative of f with respect to its input g(x), and multiply by the derivative of g(x) with respect to x.

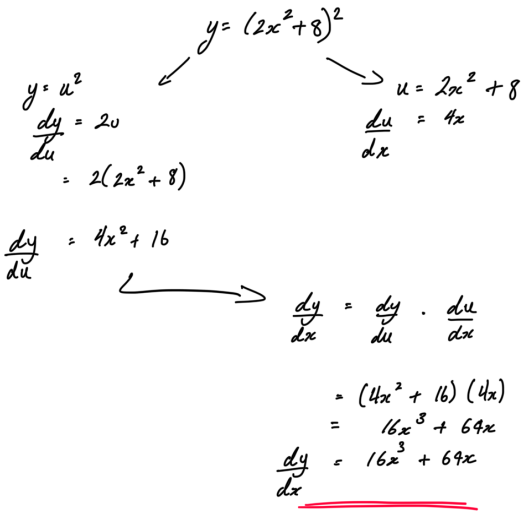

Figure 6 - Worked out example of the Chain Rule

Fig 6. show a worked example of the Chain Rule. That function can be split into 2 different functions, the inner function which is the items in parenthesis and the outer function which is the inner function squared. The inner function can be written as u = $2x^2 + 8$ and the outer function would be u^2. The derivative of u would be $4x$, and the derivative of y would be $2u$. From the inner function we know that u = $2x^2 + 8$, so $2u$ can be written as $4x^2 + 16$. Therefore to differentiate y with respect to x, we need to multiply the two derivatives, $(4x^2 + 16)(4x)$, which would be $16x^3 + 64x$.

During training of a neural network we need to minimise the loss by modifying the weights, this is essentially $\frac{dloss}{dweights}$. Since the loss depends on weights through many layers, such as

the Convolutional and Pooling layers, the chain rule is used. In a basic neural network the input X is passed to a convolution Z, which goes to an activation A, to find the loss L. This can be written as the following:

$$\frac{dL}{dW} = \frac{dL}{dA} \cdot \frac{dA}{dZ} \cdot \frac{dZ}{dW}$$

To compute these terms in practice, we look at each component's role in the gradient flow. Gradient flow is how the gradients move backwards throughout the neural network during backpropagation to enable previous layers to learn. The first term $\frac{dL}{dA}$ is the upstream gradient, this is the error signal passed back from the subsequent layer. The second term $\frac{dA}{dZ}$ is the local derivative, which is the softmax activation function. The local derivative is how much a gradient changes as its input changes. Since this, along with cross-entropy, calculates the loss, it is how much the output changes by changing the input. The final term $\frac{dZ}{dW}$ is the result of the convolution where a filter W is applied to the input X. The local derivative with respect to the weights is the input itself. Similar to the process in which the weights are updated the bias is updated the same way.

This entire sequence of operations is organized as a computational graph, a structural map that tracks every calculation from input to loss. By recording these steps, the graph allows the network to automatically navigate backward, applying the local derivatives at each node to ensure the gradient flow correctly reaches every weight and bias in the system. In a computational graph, the loss would be the final node. The graph starts from this node then looks at the edges connected to it and passes the gradient $\frac{dL}{dA}$ back to the previous node. The graph ensures that the upstream gradient arrives at the correct node to be multiplied. In the graph, the Convolution node has multiple incoming edges: the input X and the parameters W and b. The computational graph uses the chain rule to split the incoming gradient flow, sending a version of it to each branch. This is how the graph coordinates the simultaneous update of millions of parameters without getting them mixed up.

Fig 7 shows a worked out example of backpropagation using a computational graph, and how the weights are updated. In this example L takes parameters a, b and c to compute the loss, where $L = c(a + 2b)$. Further logic is applied to make this a composite function, the inner function is $e = a + d$, where $d = 2b$. During the forward pass, marked in Green, d, e and L are calculated using the inputs marked in Blue. The derivatives tell us how much a small change in input a can affect L, so this is also done to b and c. Using the computational graph we can see that the derivative of L with respect to c is e, for a and b it can be determined using the chain rule, these are the equations marked in Yellow. Next, using partial differentiation, where we treat all other variables as constants. Each derivative is taken with respect to one variable at a time, so linear terms differentiate to their coefficients.

During backpropagation, $\frac{dL}{dL}$ would be 1. $\frac{dL}{dc}$ from the partial differentiation we know that it is e, during the forward pass e was calculated to be 5, so e is updated to 5. Working backwards, $\frac{dL}{de}$ = c, so that is also updated as marked in Red.

Using partial differentiation we know as well that $\frac{de}{dd}$ = 1, so to find the derivative of L with respect to d is $\frac{dd}{db}$ = $\frac{dL}{de}$. $\frac{de}{dd}$. Plugging in the values, d is updated to -2, and using the chain rule we know that $\frac{dL}{db}$ = $\frac{dL}{de}$. $\frac{de}{dd}$. $\frac{dd}{db}$, using the already calculated values this is equal to $-\ 2.1.2\ =\ -\ 4$. Finally, $\frac{dL}{da}$ = $\frac{dL}{de}$. $\frac{de}{da}$ so substituting the values, the final equation is $-\ 2.1\ =\ -\ 2$.

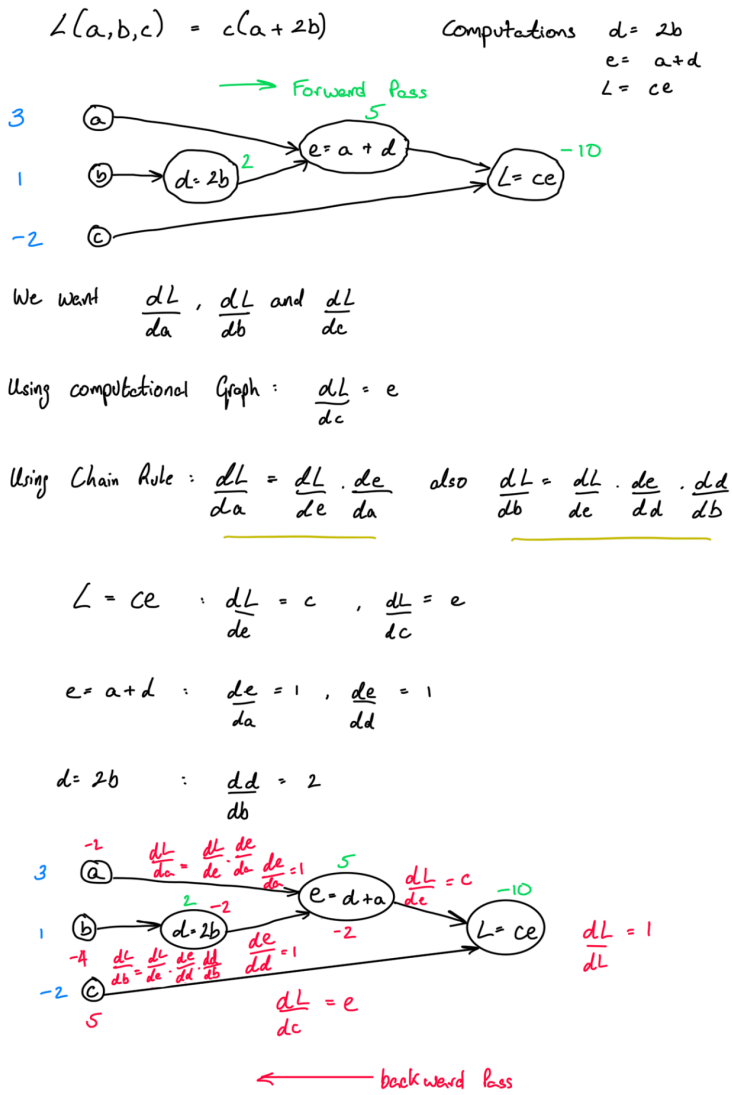

Figure 7 - Worked out example of backpropagation using computational graph

2. Quick Reference

Softmax - Mathematical function converting vectors of numbers into probabilities.
Cross-entropy - Function to determine how close probabilities are to actual labels.

Common misconception that the early works of Yann LeCun's zip code paper used Softmax + Cross-entropy. The early model used Mean Squared Error to calculate the loss, the first landmark paper to use Softmax + Cross-entropy was the paper which introduced LeNet-5 by Yann LeCun.

Forward Pass - Image being passed through the network.
Backpropagation - Optimisation of Weights and Biases to minimise loss.
Chain Rule - Easy way to find the derivative of composite functions.
Local Derivative - How much a gradient changes as its input changes.
Upstream Gradient - Error signal passed back from the subsequent layer.

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

Figure 2 - Softmax Function

$$\mathrm{CrossEntropy}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{i=1}^{n} y_i \log(\hat{y}_i)$$

Figure 3 - Cross-entropy loss function

$$\frac{d}{dx}[f\Big(g(x)\Big)] = f'\Big(g(x)\Big)g'(x)$$

Figure 5 - Chain Rule

3.Key Papers

Backpropagation applied to handwritten zip code recognition

LeCun, Y., Boser, B., Denker, J.S., Henderson, D., Howard, R.E., Hubbard, W. and Jackel, L.D., 1989. Backpropagation applied to handwritten zip code recognition. *Neural computation*, *1*(4), pp.541-551.

https://d1wqtxts1xzle7.cloudfront.net/47948178/lecun-89e-libre.pdf?1470843507=&response-content-disposition=inline%3B+filename%3DBackpropagation_Applied_to_Handwritten_Z.pdf&Expires=1777294498&Signature=Kdczwe~yQPc4NMo-W-dpPB2n8kzs83aRj6EIZiD2r0ry-0XlxsHp86H3Kgcp-W9uHBzY1su6C4Px3mVW9v-bUE97eIviebWq4Aftlkwpz9Gnlf6TwZJl0zvpFlOVVy1cV-~k8PbVTJYx2A2MwTNeEvNXRb1utudRYWPjKGwMRYNQ5sy2mLmLAmidntMGPodWU7IqC-2EZrRE-tt40qHWE2R44um3fC2D3rCBR9iHOPBuRkOwHlrS4Dv3tHSZMejf4cvXVva7139P8f7EVCRsH~OJzfB1cJeKNH49cGnWeYGGjmw0zOCEETs4j8INIpgQ~zcSLHln5pyLdHPt1XwLBA__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA

Demonstrated the first highly successful practical application of CNNs. LeCun argued that traditional fully connected networks were inefficient for image processing because they ignored the spatial topology of data. Instead, he proposed a multilayered architecture using local receptive fields, shared weights, and spatial sub-sampling. These features allowed the network to automatically learn shift-invariant visual features directly from raw pixels, eliminating the need for manual feature engineering. Trained on a dataset of handwritten digits from the US Postal Service, the system achieved a remarkable 1% error rate. This work laid the fundamental groundwork for modern computer vision and deep learning. LeCun's work proved backpropagation wasn't just a theoretical curiosity, but a scalable solution for complex, high-dimensional data. By applying the chain rule across a deep architecture of shared weights, he showed that errors could be effectively passed back to tune thousands of parameters simultaneously. This turned the chain rule from a calculus identity into the backbone of practical gradient-based learning.

Gradient-based learning applied to document recognition

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P., 2002. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), pp.2278-2324.

https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=726791&casa_token=vbs0u3u83loAAAAA:-QPZgTcBFRUnU3XTD4Fmm3ymlOS84oobqp_FlivJs2_r5rRkXtoLHUQpOGcxvV4CPRu6kImd8g

Established the now-standard pattern of alternating convolutional and pooling layers to extract hierarchical features. To benchmark this system, the authors introduced the MNIST dataset, which provided a clean, standardized collection of handwritten digits that became the gold standard for computer vision testing. Crucially, the transition from Mean Squared Error to a Softmax output paired with Cross-Entropy loss transformed the training process; this probabilistic approach ensured more robust gradients during backpropagation, allowing the chain rule to more effectively tune the network's weights and significantly improving classification accuracy over previous iterations. This paper serves as the practical bridge between multivariable calculus and modern probability. While the Chain Rule defines how to calculate derivatives, the pairing of Softmax and Cross-Entropy provides the ideal gradient for those calculations to follow. By using this combination, LeCun demonstrated that the error signal becomes significantly more intuitive and the gradient for each output neuron effectively becomes the simple difference between the predicted probability and the actual

target. This prevents the gradients from prematurely shrinking to zero, ensuring that backpropagation remains an efficient tool for training.

Highly Efficient Forward and Backward Propagation of Convolutional Neural Networks for Pixelwise Classification

Li, H., Zhao, R. and Wang, X., 2014. Highly efficient forward and backward propagation of convolutional neural networks for pixelwise classification. *arXiv preprint arXiv:1412.4526*.

https://arxiv.org/pdf/1412.4526

Marks a crucial era of computational refinement for the chain rule. By identifying and eliminating redundant calculations in pixelwise classification, the authors demonstrated that the standard backpropagation algorithm could be accelerated by three orders of magnitude. This research highlights that the scalability of deep learning wasn't just dependent on better math, but on optimizing the implementation of the chain rule to align with hardware strengths. It provides a vital counterpoint to modern critiques of backpropagation, proving that the traditional backward pass still has untapped potential for extreme efficiency when the underlying data flow is restructured. By introducing d-regularly sparse kernels, they re-engineered the way the Chain Rule is physically computed on a GPU, eliminating these redundancies to achieve a staggering 1,500x speedup without changing the final mathematical result.

The Forward-Forward Algorithm: Some Preliminary Investigations

Hinton, G., 2022. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, *2*(3), p.5.

https://www.cs.toronto.edu/~hinton/FFA13.pdf

The Forward-Forward algorithm, introduced by Geoffrey Hinton in 2022, proposes a radical departure from the traditional deep learning paradigm. Instead of using a forward pass to generate predictions and a backward pass to distribute error signals, FF uses two forward passes. The positive pass uses real data and adjusts weights , while the negative pass uses fake or corrupted data and adjusts weights to show the network what not to do. Each layer learns locally to distinguish between the two, meaning the network doesn't need to store every activity level to calculate a global gradient later. This algorithm suggests moving toward a system where layers learn independently without needing to pass gradients through the entire architecture. This shift highlights that while the chain rule was the catalyst for the deep learning revolution, the next frontier may be defined by escaping its structural rigidities.